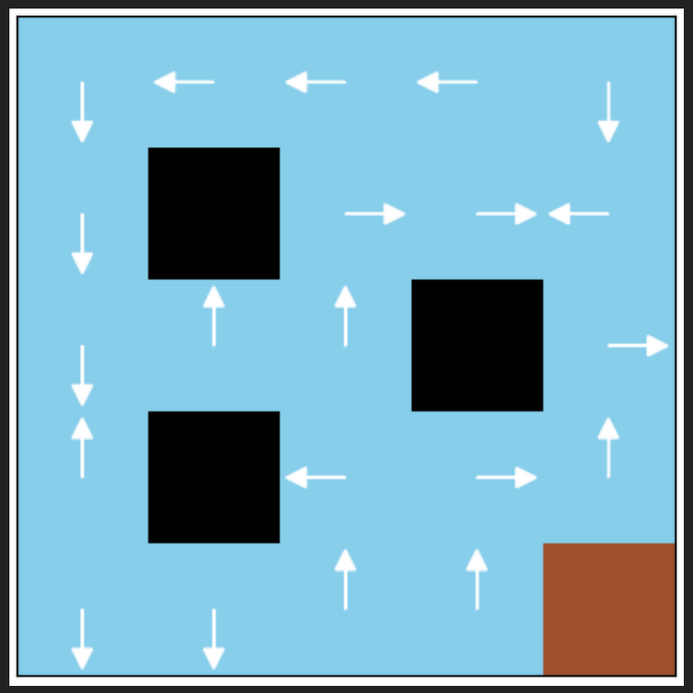
3

# EJEMPLO DE ITERACIÓN DE VALOR

# EJEMPLO – ITERACIÓN DE VALOR

- Ejemplo: Navegación de un Barco en un Océano
- El océano está dividido en una cuadrícula de celdas.
- Algunas celdas son agua navegable, otras son rocas o islotes (obstáculos), y una celda es el muelle (destino final).
- El objetivo del barco es llegar al muelle lo más rápido posible evitando las rocas.



**Value Iteration, for estimating $\pi \approx \pi_*$**

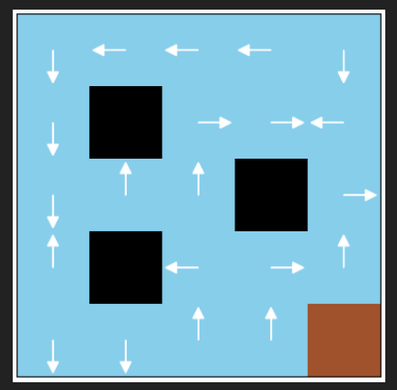
Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(terminal) = 0$

Loop:
| $\Delta \leftarrow 0$
| Loop for each $s \in \mathcal{S}$:
| $\quad v \leftarrow V(s)$
| $\quad V(s) \leftarrow \max_a \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma V(s')\big]$
| $\quad \Delta \leftarrow \max(\Delta, |v - V(s)|)$
until $\Delta < \theta$

Output a deterministic policy, $\pi \approx \pi_*$, such that
$\pi(s) = \arg\max_a \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma V(s')\big]$

# EJEMPLO – ITERACIÓN DE VALOR

- Estados: Se representan por cada celda navegable en la cuadrícula del océano. El muelle es el estado terminal.
- Acciones: El barco puede moverse en cuatro direcciones: Norte, Sur, Este, Oeste. Si el barco intenta moverse hacia una roca o fuera de los límites del océano, permanece en su celda actual.
- Recompensas:
  - ✓ -1: Por cada movimiento en una celda navegable que no sea el muelle.
  - ✓ 0: Al llegar al muelle.
- Factor de Descuento (γ): 0.9 (se busca llegar al muelle rápidamente).
- Probabilidad de Transición (p): 1 (para simplificar cálculos).



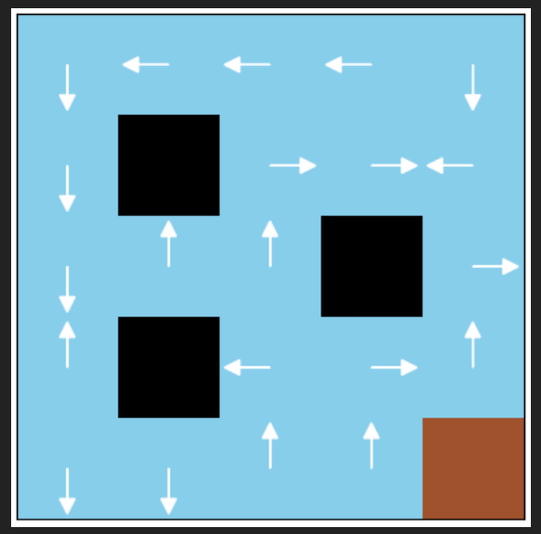
# EJEMPLO – ITERACIÓN DE VALOR

- El proceso de iteración de valor lo analizaremos en 3 pasos:

  ✓ Paso 1: Inicialización

  ✓ Paso 2: Evaluación de la Política

  ✓ Paso 3: Extracción de la Política Óptima



# EJEMPLO – ITERACIÓN DE VALOR

- Paso 1 de 3: Inicialización.

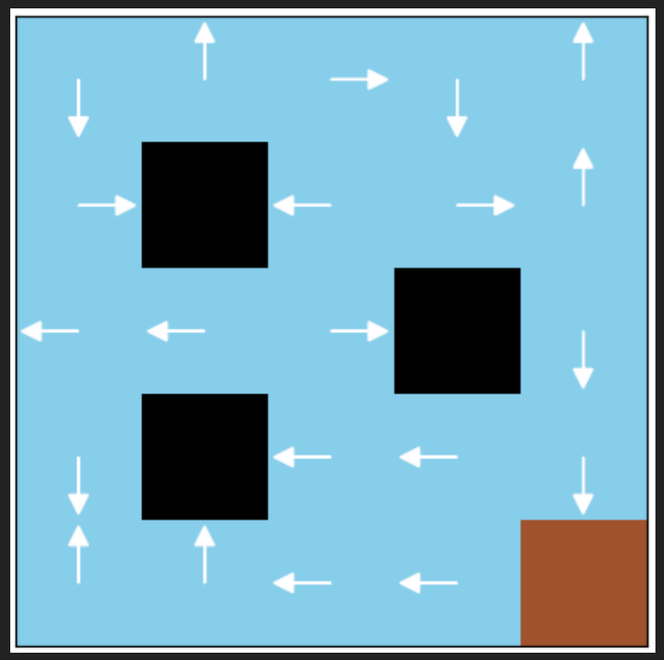
✓ Entorno: Océano de 5x5, muelle en (4, 4), rocas en (1, 1), (2, 3), (3, 1).

✓ Factor de Descuento (γ): 0.9

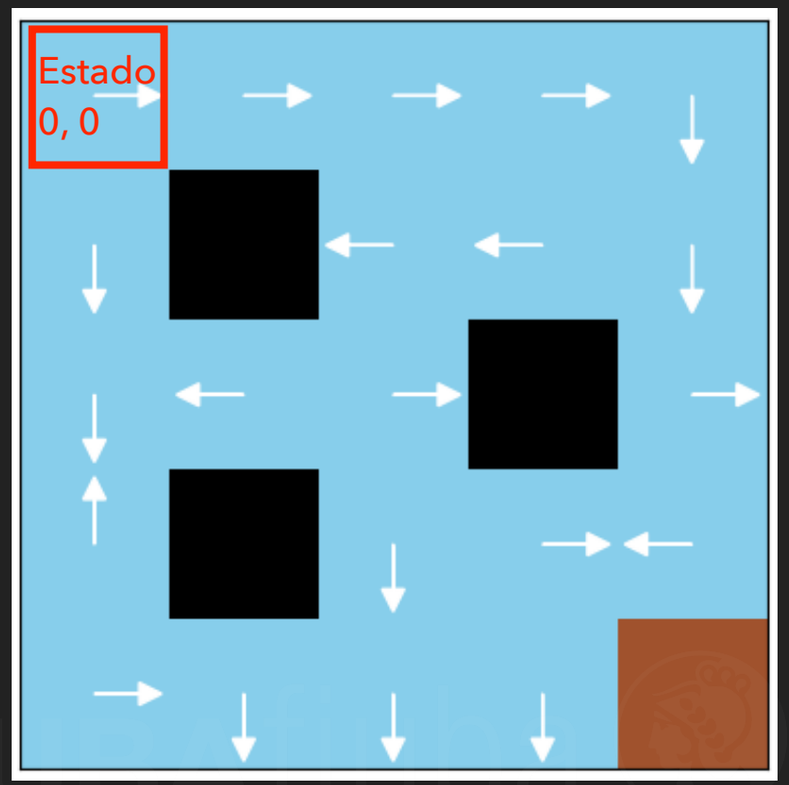
✓ Función de Valor Inicial V(s): 0 para todas las celdas.

✓ Estado Inicial: (0, 0) ubicada en la esquina superior izquierda del gridworld.

✓ Tolerancia o Umbral Theta: 0.1



# EJEMPLO – ITERACIÓN DE VALOR




- Paso 2 de 3: Evaluación de la política.
- Iteración 1 (loop):

  0. $\Delta = 0$

  1. Estado (0, 0) (Iterar entre estados navegables)

     2. v = V(0, 0) = 0

     3. Calculamos el valor de cada acción:

        ✓ Norte: R(0, 0, Norte) + γ * V(0, 0) = -1 + 0.9 * 0 = -1

        ✓ Sur: R(0, 0, Sur) + γ * V(1, 0) = -1 + 0.9 * 0 = -1

        ✓ Este: R(0, 0, Este) + γ * V(0, 1) = -1 + 0.9 * 0 = -1

        ✓ Oeste: R(0, 0, Oeste) + γ * V(0, 0) = -1 + 0.9 * 0 = -1

        ✓ V(0, 0) = max(-1, -1, -1, -1) = -1

        ✓ Δ = max(Δ, |v - V(s)| = max(0, |0 - (-1)| = 1

        ✓ 4. Pasar al estado siguiente (0, 1) hasta completar el gridworld

**Value Iteration, for estimating $\pi \approx \pi_*$**

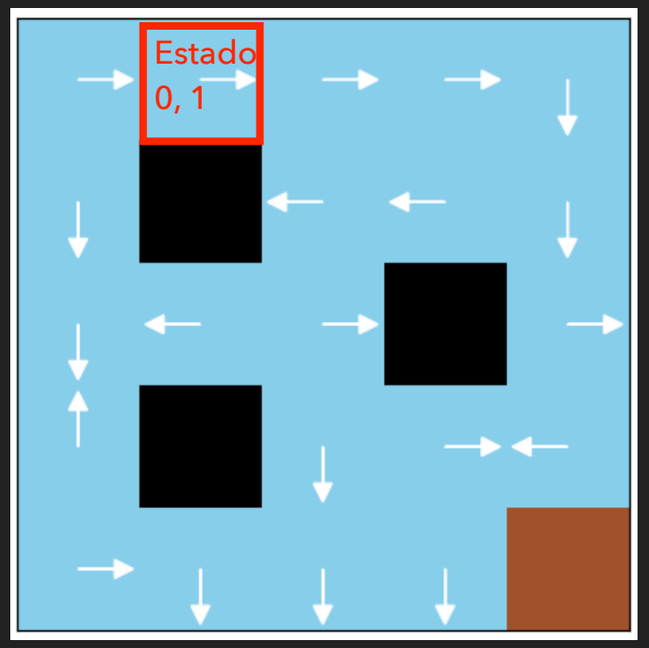
Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(terminal) = 0$

Loop:
| $\Delta \leftarrow 0$
| Loop for each $s \in \mathcal{S}$:
| $\quad v \leftarrow V(s)$
| $\quad V(s) \leftarrow \max_a \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma V(s')\big]$
| $\quad \Delta \leftarrow \max(\Delta, |v - V(s)|)$
until $\Delta < \theta$

Output a deterministic policy, $\pi \approx \pi_*$, such that
$\pi(s) = \arg\max_a \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma V(s')\big]$

- Δ < 0.1?
  En este punto, cualquier acción es igualmente buena (o igualmente mala). Independientemente de ello continuamos con el siguiente estado (0, 1).

# EJEMPLO – ITERACIÓN DE VALOR




- Paso 2 de 3: Evaluación de la política.
- Iteración 1 (loop):

  0. $\Delta = 0$

  1. Estado (0, 1) (Iterar entre estados navegables)

     2. v = V(0, 1) = 0 (el único estado que se cambió fue (0, 0) en el loop anterior)

     3. Calculamos el valor de cada acción:

        ✓ Norte: R(0, 1, Norte) + γ * V(0, 1) = -1 + 0.9 * 0 = -1

        ✓ Sur: R(0, 1, Sur) + γ * V(0, 1) = -1 + 0.9 * 0 = -1

        ✓ Este: R(0, 1, Este) + γ * V(0, 2) = -1 + 0.9 * 0 = -1

        ✓ Oeste: R(0, 1, Oeste) + γ * V(0, 0) = -1 + 0.9 * (-1) = -1.9

        ✓ V(0, 1) = max(-1, -1, -1, -1.9) = -1

        ✓ Δ = max(Δ, |v - V(s)| = max(1, |0 - (-1)| = 1

        ✓ 4. Pasar al estado siguiente (0, 2) hasta completar el gridworld

**Value Iteration, for estimating $\pi \approx \pi_*$**

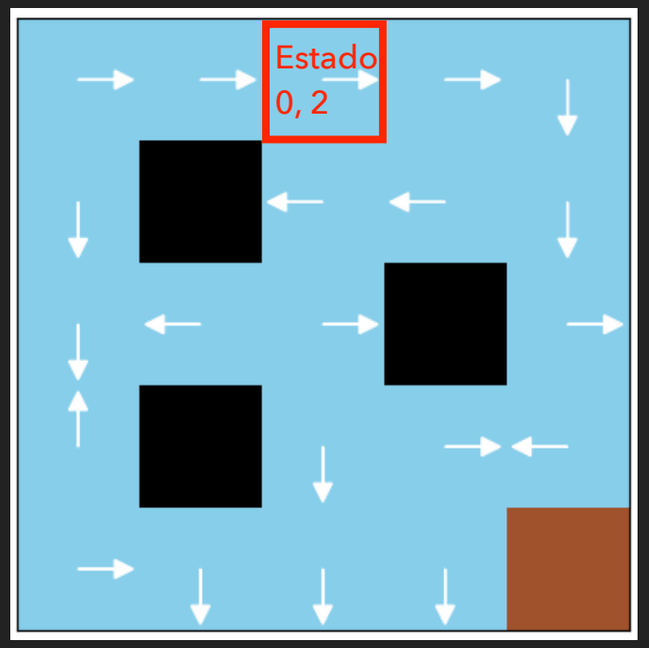
Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(terminal) = 0$

Loop:
| $\Delta \leftarrow 0$
| Loop for each $s \in \mathcal{S}$:
| $\quad v \leftarrow V(s)$
| $\quad V(s) \leftarrow \max_a \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma V(s')\big]$
| $\quad \Delta \leftarrow \max(\Delta, |v - V(s)|)$
until $\Delta < \theta$

Output a deterministic policy, $\pi \approx \pi_*$, such that
$\pi(s) = \arg\max_a \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma V(s')\big]$

- $\Delta < 0.1$?
  En este punto, cualquier acción entre Norte, Sur y Este es igualmente buena. Independientemente de ello continuamos con el siguiente estado (0, 2).

# EJEMPLO – ITERACIÓN DE VALOR




- Paso 2 de 3: Evaluación de la política.
- Iteración 1 (loop):

  0. $\Delta = 0$

  1. Estado (0, 2) (Iterar entre estados navegables)

     2. v = V(0, 2) = 0 (el único estado que se cambió fue (0, 1) en el loop anterior)

     3. Calculamos el valor de cada acción:

        ✓ Norte: R(0, 2, Norte) + γ * V(0, 2) = -1 + 0.9 * 0 = -1

        ✓ Sur: R(0, 2, Sur) + γ * V(1, 2) = -1 + 0.9 * 0 = -1

        ✓ Este: R(0, 2, Este) + γ * V(0, 3) = -1 + 0.9 * 0 = -1

        ✓ Oeste: R(0, 2, Oeste) + γ * V(0, 1) = -1 + 0.9 * (-1) = -1.9

        ✓ V(0, 2) = max(-1, -1, -1, -1.9) = -1

        ✓ Δ = max(Δ, |v - V(s)| = max(1, |0 - (-1)| = 1

        ✓ 4. Pasar al estado siguiente (0, 3) hasta completar el gridworld

**Value Iteration, for estimating $\pi \approx \pi_*$**

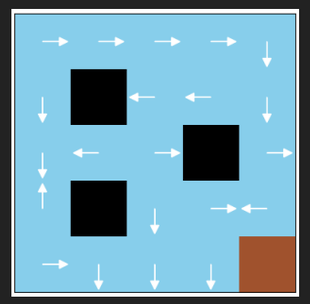
Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(terminal) = 0$

Loop:
| $\Delta \leftarrow 0$
| Loop for each $s \in \mathcal{S}$:
| $\quad v \leftarrow V(s)$
| $\quad V(s) \leftarrow \max_a \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma V(s')\big]$
| $\quad \Delta \leftarrow \max(\Delta, |v - V(s)|)$
until $\Delta < \theta$

Output a deterministic policy, $\pi \approx \pi_*$, such that
$\pi(s) = \arg\max_a \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma V(s')\big]$

- $\Delta < 0.1$?
  En este punto, cualquier acción entre Norte, Sur y Este es igualmente buena. Independientemente de ello continuamos con el siguiente estado (0, 3).

# EJEMPLO – ITERACIÓN DE VALOR



- Paso 2 de 3: Evaluación de la política.
- Si continuamos iterando alcanzamos la iteración 9 en donde $\boxed{\Delta < 0.1}$, es decir queda por debajo del umbral. Ahí finaliza el proceso de Evaluación de la política. En el siguiente paso (Paso 3 de 3) se extrae la política optima.
- Iteración 8 y 9 (loop):

```
Iteración 8: Delta = 0.47829690000000014
[-5.6953279 -5.217031  -4.68559   -4.0951    -3.439    ]
[-5.217031  0.       -4.0951   -3.439    -2.71    ]
[-4.68559 -4.0951  -3.439    0.      -1.9    ]
[-4.0951  0.     -2.71   -1.9    -1.    ]
[-3.439 -2.71  -1.9   -1.     0.   ]
Iteración 9: Delta = 0
[-5.6953279 -5.217031  -4.68559   -4.0951    -3.439    ]
[-5.217031  0.       -4.0951   -3.439    -2.71    ]
[-4.68559 -4.0951  -3.439    0.      -1.9    ]
[-4.0951  0.     -2.71   -1.9    -1.    ]
[-3.439 -2.71  -1.9   -1.     0.   ]
```

**Value Iteration, for estimating $\pi \approx \pi_*$**

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(terminal) = 0$

Loop:
| $\Delta \leftarrow 0$
| Loop for each $s \in \mathcal{S}$:
| $\quad v \leftarrow V(s)$
| $\quad V(s) \leftarrow \max_a \sum_{s',r} p(s',r|s,a)\big[r + \gamma V(s')\big]$
| $\quad \Delta \leftarrow \max(\Delta, |v - V(s)|)$
until $\Delta < \theta$

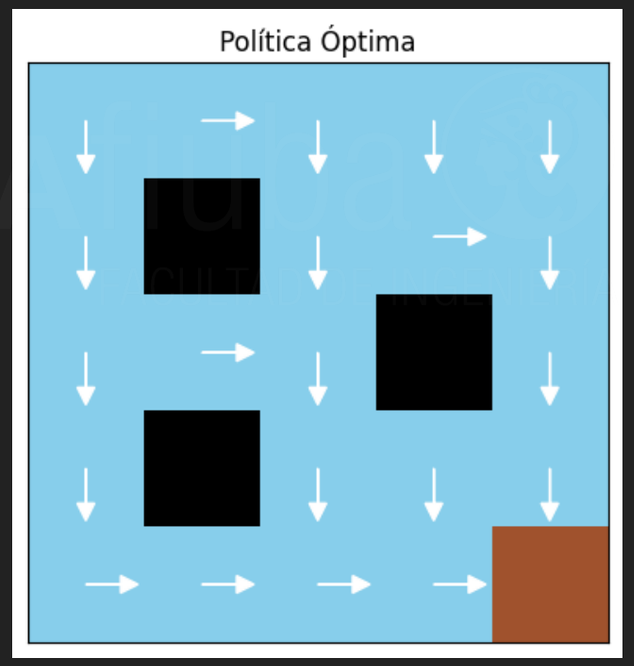
Output a deterministic policy, $\pi \approx \pi_*$, such that
$\pi(s) = \operatorname{argmax}_a \sum_{s',r} p(s',r|s,a)\big[r + \gamma V(s')\big]$

# EJEMPLO – ITERACIÓN DE VALOR

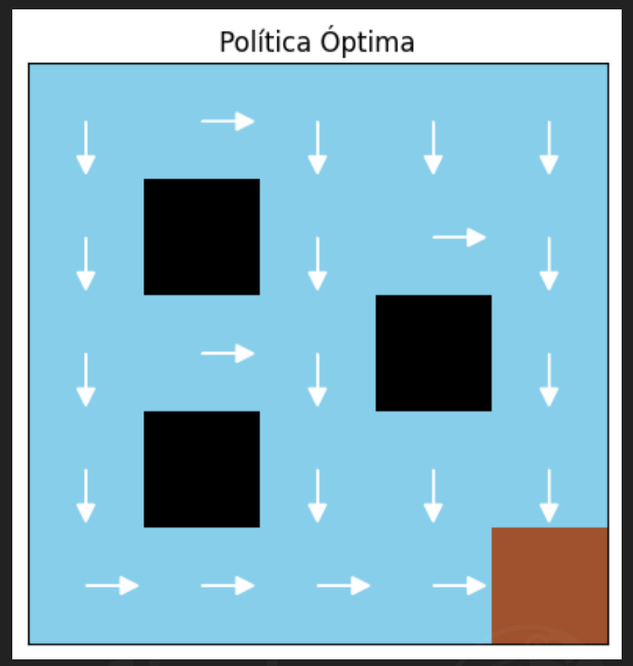
- Paso 3 de 3: Extracción de la Política Óptima.
- Objetivo: Determinar la política óptima $\pi^*(s)$ seleccionando, para cada estado s, la acción que maximiza el valor esperado de retorno utilizando la función de valor óptima V*(s) obtenida en la iteración final.
- Ecuación de la politica optima:

$$\pi^*(s) = \text{argmax_a} \; [\; R(s, a, s') + \gamma * V^*(s') \;]$$

- Esto significa que para cada celda s, la política óptima $\pi(s)$ elige la acción a (Norte, Sur, Este, Oeste) que maximiza el valor esperado de la celda resultante s'.



# EJEMPLO – ITERACIÓN DE VALOR

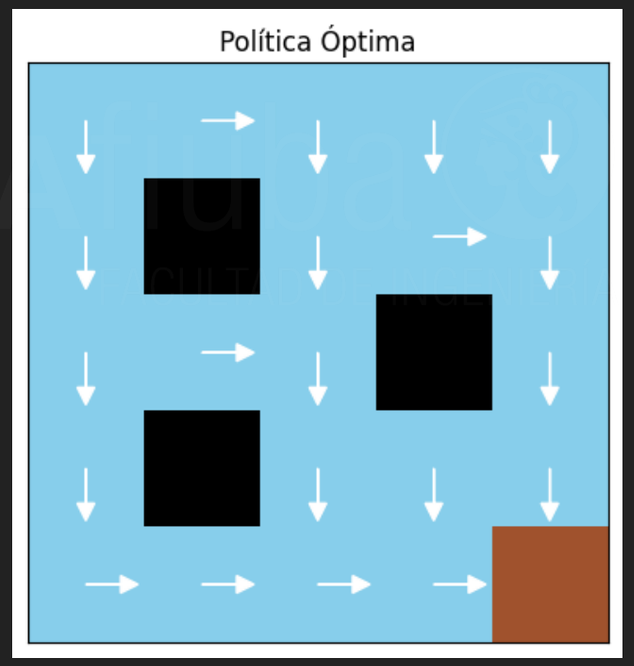



- Paso 3 de 3: Extracción de la Política Óptima.
- Estado Actual: (0,0)
- V*(0,0) = -5.6953279
- Calculamos el valor esperado de cada acción desde (0,0):
  - ✓ R(0,0, Norte) + γ * V*(0,0) = -1 + 0.9 * (-5.6953279) = -6.12579511
  - ✓ R(0,0, Sur) + γ * V*(1,0) = -1 + 0.9 * (-5.217031) = -5.6953279
  - ✓ R(0,0, Este) + γ * V*(0,1) = -1 + 0.9 * (-5.217031) = -5.6953279
  - ✓ R(0,0, Oeste) + γ * V*(0,0) = -1 + 0.9 * (-5.6953279) = -6.12579511
- max(-6.12579511, -5.6953279, -5.6953279, -6.12579511) = -5.6953279
- La política para (0,0) es: SUR o ESTE (ambas con -5.6953279). Elegimos SUR.

.UBAfiuba FACULTAD DE INGENIERÍA

# EJEMPLO – ITERACIÓN DE VALOR

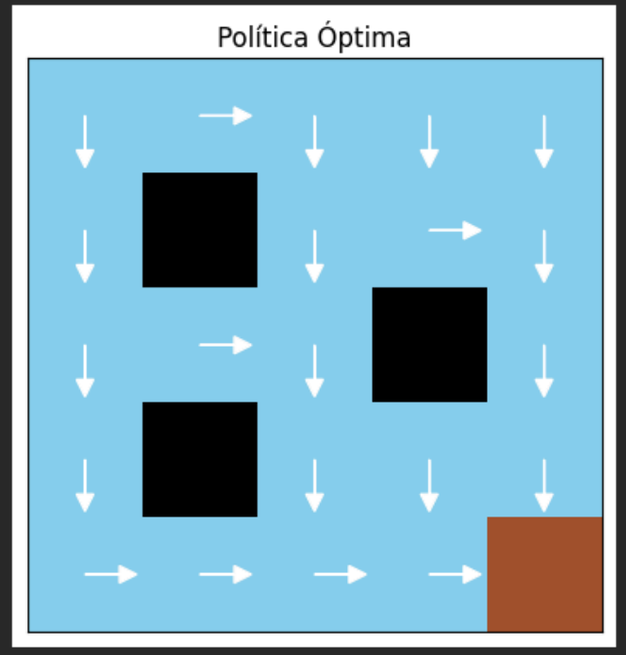
- Paso 3 de 3: Extracción de la Política Óptima.
- Estado Actual: (0,1)
- V*(0,1) = -5.217031
- Calculamos el valor esperado de cada acción desde (0,1):
  - ✓ R(0,1, Norte) + γ * V*(0,1) = -1 + 0.9 * (-5.217031) = -5.6953279
  - ✓ R(0,1, Sur) + γ * V*(0,1) = -1 + 0.9 * (-5.217031) = -5.6953279
  - ✓ R(0,1, Este) + γ * V*(0,2) = -1 + 0.9 * (-4.68559) = -5.217031
  - ✓ R(0,1, Oeste) + γ * V*(0,0) = -1 + 0.9 * (-5.6953279) = -6.12579511




- max(-5.6953279, -5.6953279, -5.217031, -6.12579511) = -5.217031
- La política para (0,1) es ESTE. Continuar iterando cada estado…

# EJEMPLO – ITERACIÓN DE VALOR

- Paso 3 de 3: Extracción de la Política Óptima.
- Después de calcular la acción óptima para cada estado navegable, obtenemos la siguiente política óptima:




```
Política Óptima:
(0, 0): Sur  (0, 1): Este  (0, 2): Sur  (0, 3): Sur  (0, 4): Sur
(1, 0): Sur  (1, 1): Roca  (1, 2): Sur  (1, 3): Este  (1, 4): Sur
(2, 0): Sur  (2, 1): Este  (2, 2): Sur  (2, 3): Roca  (2, 4): Sur
(3, 0): Sur  (3, 1): Roca  (3, 2): Sur  (3, 3): Sur  (3, 4): Sur
(4, 0): Este  (4, 1): Este  (4, 2): Este  (4, 3): Este  (4, 4): Muelle
```

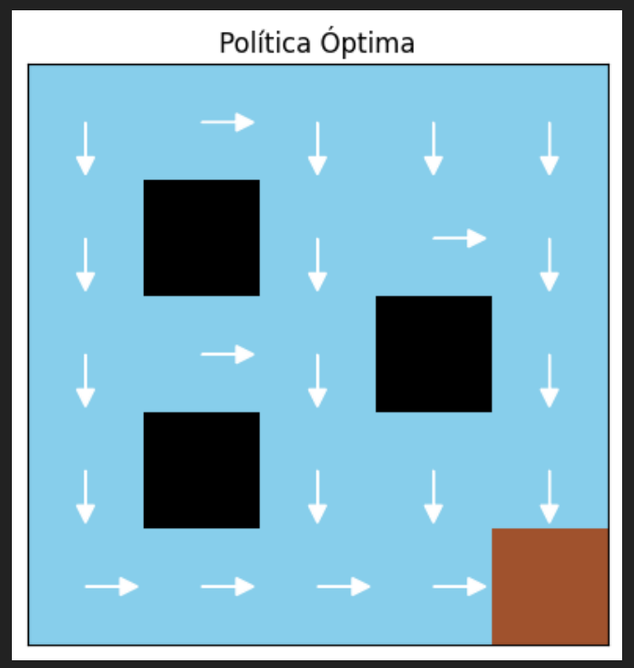
# EJEMPLO – ITERACIÓN DE VALOR

- Observaciones

✓ Con cada iteración, los valores de los estados (celdas) se ajustan y se vuelven más negativos. Esto es porque estamos penalizando cada movimiento con una rcompensa de -1.

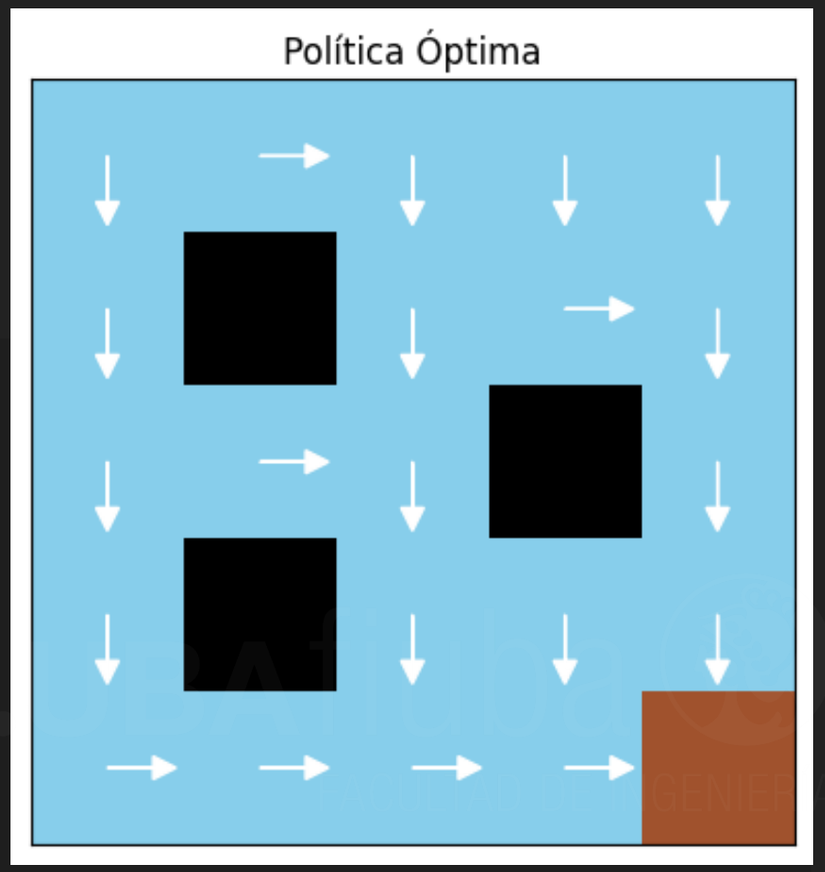
✓ Después de varias iteraciones, la política (la acción que el barco elige en cada estado) comenzará a converger hacia la ruta óptima al muelle. Por ejemplo, el barco comenzará a aprender a evitar las rocas y a moverse hacia el este y el sur para llegar al muelle.

✓ Este proceso de iteración continúa hasta que los valores de los estados dejen de cambiar significativamente. En este punto, hemos encontrado la función de valor óptima y podemos extraer la política óptima.



# EJEMPLO – ITERACIÓN DE VALOR

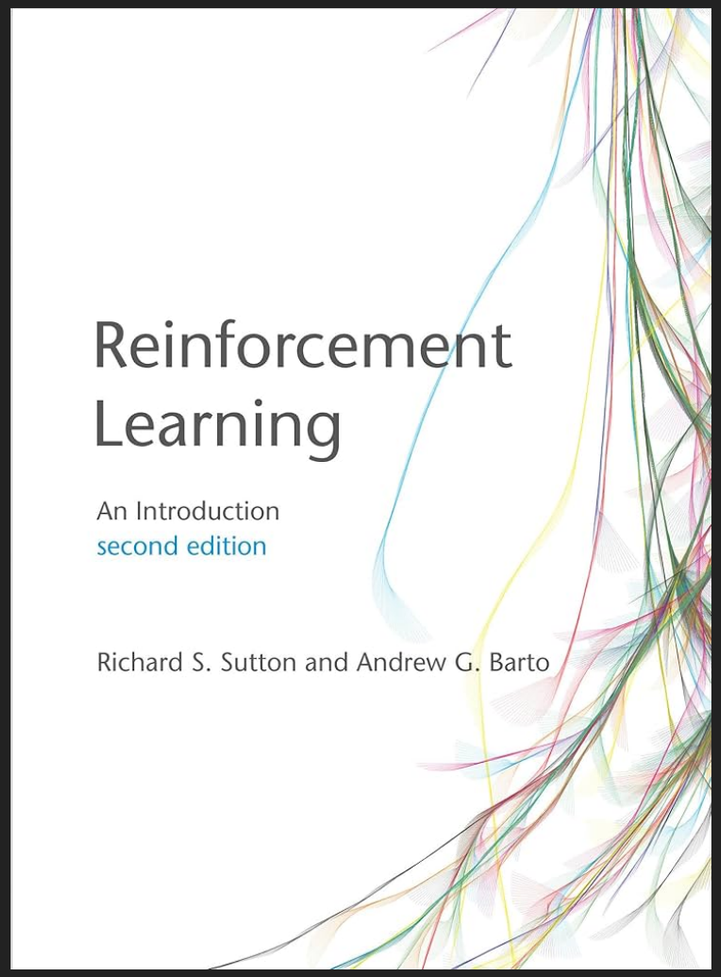
- Implementación en Python




```python
# algoritmo de iteracion de valor
iteracion = 0
while True:
    iteracion += 1
    delta = 0  # para verificar la convergencia

    # copia de la nueva finco de valor v(s)
    funcion_valor_anterior = funcion_valor.copy()

    for fila in range(LADO_OCEANO):
        for columna in range(LADO_OCEANO):
            if es_roca(fila, columna):
                continue  # no se actualizan las rocas
            if (fila, columna) == MUELLE:
                continue # no actualiza el valor del muelle
            # aqui se calcula el valor de cada celda con la ecuación de Bellman
            valores_acciones = []
            for accion in ACCIONES:
                nueva_fila, nueva_columna = transicion(fila, columna, accion) # transicion me devuelve la nueva posicion de barco (nuevo estado)
                recompensa_inmediata = recompensa(fila, columna, accion) # recompensa devuelve 0: muelle, -1: avance, -100:colision con roca
                valores_acciones.append(recompensa_inmediata + FACTOR_DESCUENTO * funcion_valor[nueva_fila, nueva_columna]) # ecuacion de bellman

            funcion_valor[fila, columna] = max(valores_acciones)
            delta = max(delta, abs(funcion_valor[fila, columna] - funcion_valor_anterior[fila, columna]))

    print(f"Iteración {iteracion}: Delta = {delta}")
    # impresion poe consola del valor de cada celda en formato matricial
    for i in range(LADO_OCEANO):
        print(funcion_valor[i,:])
    if delta < TOLERANCIA:
        break
```

# BIBLIOGRAFIA


- R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.


.UBAfiuba
FACULTAD DE INGENIERÍA